

Le Chiamate di Sistema

Lezione 11

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Panoramica
- Introduzione alla programmazione di sistema
- System call memoria
 - (richiami)
- System call file I/O



System call gestione file I/O

Noi abbiamo finora visto le funzioni di libreria del C.

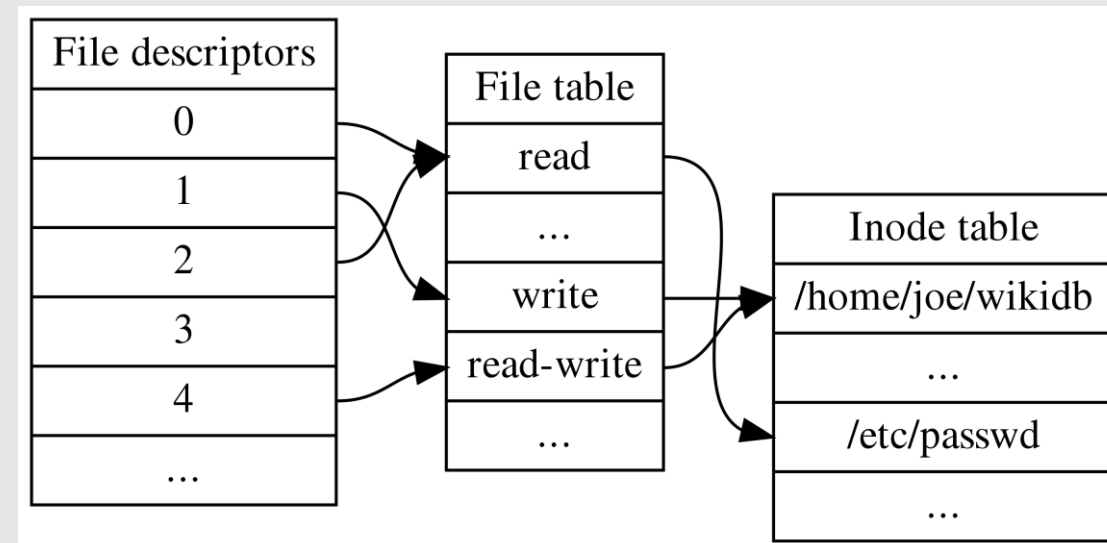


Operare sui file

- File, astrazione che descrive ogni risorsa, ad es. file su disco, dispositivi, socket, fifo
- Sequenza tipica di azioni
 1. apertura file mediante `open`; viene creato un *file descriptor*
 2. operazione su file mediante `syscall`; e.g. `write`, `read`
 3. chiusura del file mediante `close`
- Quando un processo termina, chiude automaticamente tutti i file correntemente aperti, tuttavia, **è sempre consigliabile** e buona regola di programmazione chiudere esplicitamente un file invocando la `close`

File descriptor

- E' il riferimento ad un file aperto: è un intero piccolo, generato sequenzialmente a partire dal valore 0. Per default, ogni processo, ha associato i seguenti file descriptors:
 - **0; stdin**
 - **1; stdout**
 - **2; stderr**
- Quando un file viene chiuso, il suo file descriptor viene liberato e può essere riutilizzato. È possibile aprire uno stesso file ed ottenere file descriptor diversi che puntano allo stesso file (e.g., due utenti aprono lo stesso file in lettura).



File flags associati ad un File descriptor

- **File status flags:**
 - Associati allo stato di un file
 - Sono condivisi tra tutti i file descriptor ottenuti per duplicazione da un unico file descriptor.
- **File descriptor flags:**
 - Associati al file descriptor
 - Indipendenti dal contenuto o status del file
 - Descrivono proprietà e comportamento delle operazioni effettuate sul file
 - File descriptor diversi che fanno riferimento ad uno stesso file hanno ognuno il proprio insieme di descriptor flags
 - Alcuni definiti solo per alcuni tipi di file speciali (es: fifo socket)

Rappresentazione dei Flag

- Mediante maschere di bit
 - se il bit è a 1 (settato) il flag è settato
- Per ogni flag è stata definita una maschera di bit o MACRO
 - ad es. O_RDONLY
- I flag possono essere combinati mettendo in OR bit a bit (*bitwise*) le maschere
 - ad es.
 - MACRO1 = 01000000;
 - MACRO2 = 00010000
 - MACRO1 | MACRO2 = 01010000
 - dopo l'OR entrambi i flag sono settati



File status flags: 3 categorie

- Modalità di accesso
 - read, write, read+write.
 - specificati nella open
 - non possono essere modificati una volta aperto il file
- Di apertura
 - definiscono il comportamento della open e non vengono mantenuti
- Modalità operative
 - definiscono il comportamento delle operazioni read e write.
 - Specificati nella open.
 - Possono essere modificati anche dopo l'apertura del file.



System call principali

- open
- chown
- read
- chmod
- write
- stat
- lseek
- select
- close
- ioctl
- unlink
- fnctl
- symlink
- rename
- mkdir
- rmdir
- chdir



Open ()

```
int open(const char *pathname, int flags);  
int open(const char *pathname, int flags, mode_t mode);
```

- restituisce -1 se errore altrimenti file descriptor
- parametro `flags` (file status flags); equivalente al parametro `mode` della `fopen`
 - `O_CREAT` richiede di specificare i permessi di accesso al file (ad es. 644)
- `mode` indica le modalità di creazione del file

\$ man fopen

fopen() mode	open() flags
<u>r</u>	<u>O_RDONLY</u>
<u>w</u>	O_WRONLY O_CREAT O_TRUNC
<u>a</u>	O_WRONLY O_CREAT O_APPEND
<u>r+</u>	O_RDWR
<u>w+</u>	O_RDWR O_CREAT O_TRUNC
<u>a+</u>	O_RDWR O_CREAT O_APPEND

Differenza `open` e `fopen` (APUE Ch. 5)

- `fopen` returns a *pointer* to a FILE object.
- This object is normally a structure that contains all the information required by the standard I/O library to manage the stream:
 - the file descriptor used for actual I/O,
 - a pointer to a buffer for the stream,
 - the size of the buffer,
 - a count of the number of characters currently in the buffer,
 - an error flag,
 - and the like.

Flags open

- Flags modalità accesso
 - O_RDONLY, O_WRONLY, O_RDWR
- Flags di apertura
 - O_CREAT
 - O_EXCL quando specificato insieme a O_CREAT genera errore se il file esiste già
- Flags modalità operativa
 - O_APPEND
 - O_SYNC scrittura sincrona, ovvero la call ritorna solamente quando la scrittura dei dati nel file è terminata
 - O_TRUNC se il file esiste, è un file regolare, e la modalità di accesso consente la scrittura, allora il file viene troncato dalla posizione 0

ci sono 8 flags ...
rappresentabili, ad esempio,
con una maschera a 8 bit

```
ssize_t read(int fd, void *buf,  
              size_t count)
```

- `fd` file descriptor
- `buf` puntatore ad area di memoria in cui memorizzare i byte letti
- `count` numero di byte da leggere
- restituisce -1 se errore, altrimenti il numero di byte letti (che **può essere < di count** se si raggiunge EOF)
- i codici di errore, restituiti in `errno`, sono elencati su
 - `man 2 read`

Differenza tra read e fread

- `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream)`
 - legge da uno stream di tipo `FILE` (bufferizzata)
 - dimensione del tipo di dato da leggere
- `ssize_t read(int fd, void *buf, size_t count);`
 - **non** bufferizzata
 - lavora sui byte indipendentemente dal tipo di dato in essi contenuto

`fopen` returns a pointer to a `FILE` object. This object is normally a structure that contains all the information required by the standard I/O library to manage the stream: the file descriptor used for actual I/O, a pointer to a buffer for the stream, the size of the buffer, a count of the number of characters currently in the buffer, an error flag, and the like.

```
ssize_t write(int fd, const void *buf,  
size_t count)
```

- `fd` file descriptor
- `buf` area di memoria da cui leggere i dati; dichiarata `const` per non essere modificata dalla funzione
- `count` numero di byte da scrivere
- restituisce -1 se errore o numero di byte scritti (**può essere < di `count`** se `write` interrotta, ad esempio da un segnale)
- i codici di errore, restituiti in `errno`, sono elencati su
 - `man 2 write`

```
int close(int fd);
```

- Chiude il file descriptor `fd`
 - Il file descriptor viene liberato e può essere quindi riutilizzato
- Ritorna -1 in caso di errore e 0 in caso la chiamata termini correttamente
- Nel caso venga chiuso l'ultimo file descriptor che fa riferimento ad un file rimosso, allora il file viene cancellato.
- I codici di errore, restituiti in `errno`, sono elencati su
 - `man 2 close`

Esempio

- file `file.c` della Lezione 9
- **Esercizio 1:** rifare l'Esercizio 6 lezione 9 usando le syscall `read` e `write` (aggiungere `#include <fcntl.h>` al vostro codice).
- **Esercizio 2:** realizzare un programma che legge da riga comando tre argomenti: (1) il nome di un file di input, (2) il nome di un file di output, (3) un numero naturale n . Si deve copiare il file di input nel file di output invertendo ciascun blocco di dimensione n .



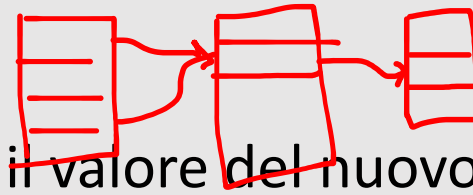
Altre system call e funzioni di libreria

- `int dup(int oldfd);`
- `int stat(const char *path, struct stat *buf);`
- `int chmod(const char *path, mode_t mode)`
- `int chown(const char *path, uid_t owner, gid_t group)`
- `int rename(const char *oldpath, const char *newpath);`
- `int mkdir(const char *pathname, mode_t mode);`
- `DIR *opendir(const char *name); //Libreria`
- `struct dirent *readdir(DIR *dirp); //Libreria`
- `int closedir(DIR *dirp); //Libreria`
- `int chdir(const char *path);`
- `int fcntl(int fd, int cmd, ... /* arg */ );`
- `int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout); //la vedremo dopo`

vediamone
alcune

- `int dup(int oldfd);`

- duplica il file descriptor `oldfd` e restituisce il valore del nuovo `fd`.
Restituisce -1 in caso di errore



- `int stat(const char *path, struct stat *buf);`

- `int fstat(int fd, struct stat *buf);`

- restituisce in `buf` le informazioni di stato del file specificato con nome file `path` o con file descriptor `fd`.
- Ritorna 0 se termina correttamente; -1 altrimenti

vedi la lista
completa dei campi
di `struct stat`
con
`man 2 stat`

```
struct stat {
    dev_t      st_dev;           /* ID of device containing file */
    ino_t      st_ino;          /* inode number */
    mode_t     st_mode;          /* file type and mode */
    nlink_t    st_nlink;         /* number of hard links */
    uid_t      st_uid;           /* user ID of owner */
    gid_t      st_gid;           /* group ID of owner */
    dev_t      st_rdev;          /* device ID (if special file) */
    off_t      st_size;          /* total size, in bytes */
    blksize_t  st_blksize;       /* blocksize for filesystem I/O */
    blkcnt_t   st_blocks;        /* number of 512B blocks allocated */
}
```



stat (cont.)

- Sono definite anche una serie di macro da utilizzare sulla struttura dati `stat (buf)` per verificare il tipo del file:
- `S_ISREG (m)` , `S_ISDIR (m)` , `S_ISCHR (m)` ,
- `S_ISBLK (m)` , `S_ISFIFO (m)` , `S_ISLNK (m)` ,
- `S_ISSOCK (m)`
- e maschere da usare per verificare i diritti di accesso (vedi prossima slide e `man 2 stat` oppure `man 7 inode`)

```
int chmod(const char *pathname, mode_t mode)
int fchmod(int fd, mode_t mode)
```

- cambia il file mode. Ritorna -1 se errore; 0 altrimenti
- mode è un numero ottale, e.g. 0755. Si possono usare le maschere predefinite (vedi man 2 chmod), tipo

- S_ISUID (04000) set-user-ID
- S_ISGID (02000) set-group-ID
- S_ISVTX (01000) sticky bit
- S_IRUSR (00400) read by owner
- ...
- S_IRGRP (00040) read by group
- ...
- S_IROTH (00004) read by others

Eventualmente
in bitwise OR

- vedi esempio dupstat.c
- Vedi dup.c, dup2.c dup3.c



```
DIR *opendir(const char *name)
struct dirent *readdir(DIR *dirp)
int closedir(DIR *dirp)
```

- Non sono system call ma funzioni di libreria (sezione 3 del man)
- Permettono di aprire una directory il cui stream è ritornato da `opendir`
- `readdir` legge il contenuto della directory (prossimo elemento disponibile) ritornando la struttura `dirent` o NULL se non ci sono più elementi

```
struct dirent {
    ino_t      d_ino;      /* Inode number */
    off_t      d_off;      /* Not an offset; see below */
    unsigned short d_reclen; /* Length of this record */
    unsigned char d_type;   /* Type of file; not supported
                           by all filesystem types */
    char        d_name[256]; /* Null-terminated filename */
};
```



- vedi esempio `directory.c`, `readdir.c`



```
int fcntl(int fd, int cmd, ...
          /* arg */ )
```

- System call - Permette di effettuare operazioni sul file descriptor `fd`, ad esempio
 - duplicazione del `fd`
 - manipolazione flag file descriptor
 - manipolazione flag di stato
 - gestione lock su `fd`
- A tali insiemi di operazioni sono associati un insieme di comandi passata con il parametro `cmd` di `fcntl`
- Gli argomenti del comando `cmd`, se presenti, si passano come parametri `arg` della syscall e vanno messi dopo `cmd`
- Il valore di ritorno di `fcntl` dipende dall'operazione effettuata
 - `man 2 fcntl` per i dettagli

fcntl - manipolazione flag di stato

- `F_GETFL(void)`
 - restituisce, come risultato di `fcntl`, file access mode e file status flag
 - e.g. `val=fcntl(fd, F_GETFL)`
- `F_SETFL(int)`
 - imposta i file status flag `O_APPEND`, `O_ASYNC`, `O_DIRECT`, `O_NOATIME`, `O_NONBLOCK`
 - e.g. `fcntl(fd, F_SETFL, O_APPEND)`
- vediamo `fcntl.c`

fcntl - lock su file

- F_SETLK, acquisisce/rilascia lock
- F_SETLKW, acquisisce/rilascia lock, bloccante
- F_GETLK, testa esistenza lock
- L'argomento è la struttura flock
- l_whence dice da dove parte l_start (> 0 < 0)
 - SEEK_SET=inizio file
 - SEEK_CUR=posizione corrente
 - SEEK_END=fine file
- l_len = 0 tutto il file viene bloccato
altrimenti vengono bloccati l_len byte
a partire da l_start

```
struct flock {  
    ...  
    short l_type; /* Type of lock: F_RDLCK,  
                  F_WRLCK, F_UNLCK */  
    short l_whence; /* How to interpret l_start:  
                   SEEK_SET, SEEK_CUR, SEEK_END */  
    off_t l_start; /* Starting offset for lock */  
    off_t l_len; /* Number of bytes to lock */  
    pid_t l_pid; /* PID of process blocking our lock  
                (set by F_GETLK and F_OFD_GETLK) */  
    ...  
};
```

vediamo fcntl2.c



fcntl - lock su file (cont'd)

- `F_SETLK`
 - acquisisce lock se `l_type = F_RDLCK` o `F_WRLCK`,
 - rilascia se `l_type = F_UNLCK`
 - restituisce -1 se un altro processo ha il lock
- `F_SETLKW`
 - come la `F_SETLK` ma se c'è un lock sul file attende (wait) che questo venga rilasciato
- `F_GETLK`, **testa esistenza lock**
 - la struttura `flock` contiene il lock che si vuole mettere
 - se il lock può essere messo `fcntl` aggiorna `l_type` a `F_UNLCK`
 - deve quindi essere seguita da `F_SETLK/LKW`

fcntl - lock su file - considerazioni

- Il lock è *advisory*, ovvero richiede cooperazione tra processi
 - tutti i processi fanno una `F_GETLK` o `F_SETLK/LKW` e osservano il risultato
 - cercare di scrivere un file sul quale un processo detiene un lock non ha l'effetto di bloccare la scrittura
- Per avere un lock *mandatory* (obbligatorio) ovvero che impedisce la scrittura o lettura richiede che il file system supporti il mandatory locking

vediamo `fcntl2.c`



```
int select(int nfd, fd_set *readfds, fd_set
*writefds, fd_set *exceptfds, struct timeval
*timeout)
```

- System call - Permette di monitorare uno o più file descriptor rimanendo in attesa che almeno uno di essi sia disponibile per effettuare l'operazione richiesta
- L'utilizzo della select consente di sincronizzare dei processi tramite il monitoraggio della disponibilità di accesso ad un file
 - `nfd` numero del file descriptor con il valore più alto aumentato di 1 tra quelli da monitorare
 - `readfds` insieme contenente i file descriptor per cui si richiede la lettura
 - `writefds` insieme contenente i file descriptor per cui si richiede la scrittura
 - `exceptfds` insieme contenente i file descriptor da controllare per le eccezioni
 - `timeout` timeout di attesa.

select – valore di ritorno

- La funzione ritorna il numero dei file descriptor disponibili per l'operazione richiesta, oppure ritorna -1 in caso di errore.
- Il valore ritornato può essere 0 se la chiamata termina per il timeout, prima che uno dei descrittori diventi disponibile.
- Al termine dell'esecuzione, la funzione aggiorna le strutture dati contenenti i file descriptor da monitorare, per indicare quali di questi è disponibile per l'operazione richiesta.

select – valori timeout

- **infinito** la chiamata è bloccante, ovvero termina quando è disponibile almeno un descrittore, oppure si genera un errore
- **intervallo definito** la chiamata ritorna se è disponibile almeno un descrittore, oppure se è scaduto il timeout, oppure se si genera un errore
- **zero** la chiamata non è bloccante, ritorna subito dopo aver controllato i descrittori. Questa modalità può essere utilizzata per effettuare *polling*.

select – insiemi dei file descriptor

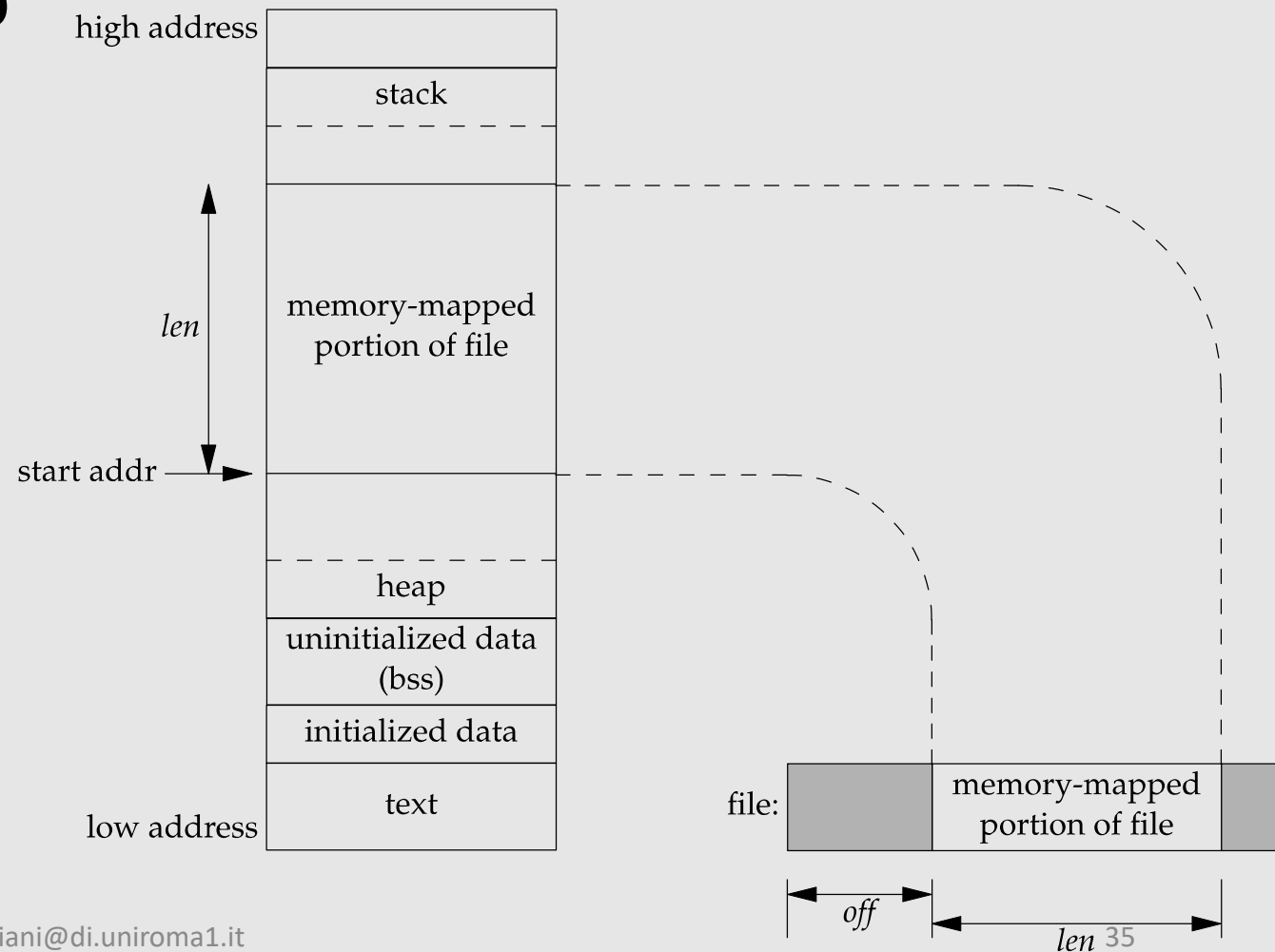
- Sono messe a disposizione quattro macro per la gestione degli insiemi dei file descriptor
 - `FD_ZERO()` svuota un insieme.
 - `FD_SET()` aggiunge un file descriptor ad un insieme
 - `FD_CLR()` rimuove (clear) un file descriptor da un insieme
 - `FD_ISSET()` testa se un file descriptor appartiene ad un insieme. Utile quando `select` ritorna
- I prototipi sono:
 - `void FD_ZERO(fd_set *set);`
 - `void FD_SET(int fd, fd_set *set);`
 - `void FD_CLR(int fd, fd_set *set);`
 - `int FD_ISSET(int fd, fd_set *set);`

Esercizio

- Progettare un programma C che:
 - legga il contenuto di una directory `dir` passata come parametro di ingresso. `dir` può contenere file e directory. Il programma deve visitare `dir` e l'albero delle sotto directory a partire da `dir` fino ad una profondità `n` data come parametro di ingresso.
 - Ogni file eseguibile deve essere copiato in una directory nominata `exec` (fuori da `dir`) e, dopo essere stato copiato, deve poter essere eseguibile solo dal proprietario.
 - I file non eseguibili, ma regolari, devono essere copiati in una directory nominata `regular`
 - [Il programma deve poter svolgere il suo compito sfruttando `p` processi che lavorano in parallelo, dove `p` è un parametro di ingresso. La scelta della politica di suddivisione del lavoro tra più processi è lasciata a voi.]

```
void *mmap (void *addr, size_t len, int
prot, int flag, int fd, off_t off )
```

- Implementa il **memory-mapped I/O**
- Mappa un file su disco in un area di memoria (buffer)
- Lettura/scrittura dal/sul buffer risultano in lettura scrittura dal/sul disco (senza accedere al disco)
- Uso:
 - `#include <sys/mman.h>`
 - Restituisce l'indirizzo di partenza della regione mappata (buffer) se OK, MAP_FAILED se errore



```
void *mmap (void *addr, size_t len, int
prot, int flag, int fd, off_t off )
```

- *addr* è l'indirizzo iniziale dell'area di memoria in cui vogliamo mappare il file. Se *addr* = NULL, il kernel sceglie per noi
- *fd* è il file descriptor (il file va aperto prima)
 - *len* il numero di byte da trasferire
 - *off* l'offset nel file

- *prot* indica il livello di protezione dell'area mappata

<i>prot</i>	Description
PROT_READ	Region can be read.
PROT_WRITE	Region can be written.
PROT_EXEC	Region can be executed.
PROT_NONE	Region cannot be accessed.

- *flag* (attributi dell'area mappata)
 - MAP_SHARED le operazioni sulla memoria modificano il file (R/W)
 - MAP_PRIVATE viene create una copia private del mapped file e solo questa viene modificata
 - MAP_FIXED dice al kernel di mappare a partire da *addr* [**NON USARE!!!!**]

Sync e de-mapping

```
#include <sys/mman.h>
```

```
int msync(void *addr, size_t len, int flags);
```

- Le modifiche ad una regione mappata (con MAP_SHARED, ovviamente) non vengono scritte immediatamente nel file – il kernel **decide** quando farlo
- Con msync comunicate al kernel che volete scrivere sul file le modifiche in memoria
- Il kernel farà sempre l'I/O in *multipli* della dimensione di una *pagina di memoria*

```
#include <sys/mman.h>
```

```
int munmap(void *addr, size_t len);
```

- Quando il processo termina la memoria viene automaticamente de-mappata, ma ciò non significa che il contenuto è scritto sul file (vedi sopra)
- Ciò non avviene se chiudo il file, quindi sempre buona regola de-mappare la memoria con munmap

msync e munmap ritornano -1 se errore



Esempio uso `mmap` (pag. 529 di APUE)



Esercizio su mmap

- Scrivere un programma C che:
 - legga un file di testo e produca in output un file di testo in cui ogni carattere minuscolo è trasformato in maiuscolo, e viceversa
- Requisiti implementativi:
 - Usare `mmap` per mappare l'intero file in memoria
 - Manipolare il testo di output direttamente in memoria (usate il flag `MAP_SHARED` in `mmap` per permettere al kernel di scrivere il file)

Domande?

